FIELD JOURNAL

Compare ordered data without losing its structure

What does it actually mean for two sequences to be similar?

Two journeys can appear more or less similar depending on which differences we measure.

00 The investigation — Which journeys are most similar?

A discrete sequence is an ordered list of events. A customer journey records those events in the order they occurred.

The same events can lead to different outcomes depending on their order.

A — Direct purchase
The customer searches, views, a product, adds it to the cart, and purchases.
Search → View → Cart → Purchase

B — Comparison shopping
The customer searches, views, compares alternatives, then adds to cart and purchases.
Search → View → Compare → Cart → Purchase

C — Cart abandonment
The customer searches, views, adds to cart, but exits instead of purchasing.
Search → View → Cart → Exit

Key takeaway
Journey B reaches the same purchase outcome as A, but takes an extra step.
Journey C follows A's first three actions, but leaves without purchasing.

Which is closer to A?

01 Representation — What does a sequence preserve?

1. Item set (ignores order and repetition)
An item set records which distinct actions occurred. It discards order and repetition.
{ Search, View, Cart, Purchase }

2. Event counts (ignores order)
A count vector records how many times each action occurs. It preserves frequency but discards order.

Search	View	Compare	Cart	Purchase
1	1	0	1	1

Counts can distinguish repeated actions (e.g., View = 3), but not their order.

3. Ordered sequence (keeps order)
An ordered sequence preserves which events occurred, how often they occurred, and their relative order.
Search → View → Cart → Purchase

4. Timestamped sequence (keeps order and timing)
A timestamped sequence preserves order and timing. Two customers can follow the same steps but take different amounts of time.
10:00 → 10:05 → 10:10 → 10:25
Same order ≠ same elapsed time.

Same actions, different order
Two journeys contain the same actions and the same counts, but in a different order. Once order is discarded, item sets and count vectors cannot recover which action happened first.

Journey 1: Search → View → Cart → Purchase
Journey 2: Search → Cart → View → Purchase

How do they compare?	
Same item set?	Yes
Same counts?	Yes
Same order?	No

They contain the same actions and the same counts, but a different order. Item sets and count vectors cannot tell which action happened first.

02 Alignment — How can sequences match when positions shift?

Alignment restores correspondence by inserting gaps so events that occur at different positions can still be compared.

The big idea
Two journeys can contain the same events in the same order, but appear different when an event is inserted, deleted, or moved. Alignment inserts gaps (-) so equivalent events line up.

Before alignment (compare by position)

A:	S	V	C	P		
B:	S	V	X	C	P	

B has an extra Compare (X), so later events are shifted when comparing by position.

Position-by-position comparison would mark C and P as mismatches.

After alignment (insert a gap)

A:	S	V	–	C	P	
B:	S	V	X	C	P	

Insert a gap in A so that S, V, C, P line up. Four events now correspond; Compare (X) is the only unmatched action.

Aligned sequences reveal the same events, even though they occur at different positions.

Why alignment matters

Compare the same events
Lets us match events even when they occur at different positions.

Measure meaningful similarity
Reveals similarity that a simple position-by-position comparison would miss.

Foundation for edit distance
Provides a position-by-position view that supports edit distance and related measures.

Two common objectives

1 Global alignment
Align the full sequences (from start to end).
Useful when we want an overall comparison (e.g., edit distance).

2 Local alignment
Find the highest-scoring matching region within two sequences.
Useful when only a portion of the sequences may be similar.

03 Edit distance — What does it cost to transform one journey into another?

Edit distance is the minimum total cost to transform one sequence into another using insertions, deletions, and substitutions.

The three operations (unit cost = 1)
Every transformation is built from three edits. Each changes the sequence in a different way.

Insertion — Add an event.
S V C P → S V X C P
Insert Compare (X) between View and Cart.
cost = 1

Deletion — Remove an event.
S V X C P → S V C P
Delete Compare (X) from B to get A.
cost = 1

Substitution — Replace an event.
P → E
Replace Purchase (P) with Exit (E).
cost = 1 (or higher)

Examples with our journeys
Under unit costs, an extra Compare step and a changed purchase outcome each cost one edit.

A → B: insert Compare

A:	S	V	C	P		
B:	S	V	X	C	P	

Edit distance = 1 (insert X).

A →C: substitute Purchase → Exit

A:	S	V	C	P	
C:	S	V	C	E	

Edit distance = 1 (substitute P with E).

Weighted costs
When some changes matter more than others, we can assign different costs to edits.

Substitute: P → E, cost = 4
>
Delete + insert: P → – → E, cost = 1 + 1 = 2

The optimizer chooses the cheaper valid transformation, even when substitution exists.

Normalized distances (similarity = 1 − normalized distance)

Pair	Edit distance	Longer length	Normalized distance	Similarity
A – B	1	5	1/5 = 0.20	0.80
A – C	1	4	1/4 = 0.25	0.75

Why normalize?
One edit can be a small change in a long journey or a large change in a short one. Dividing by the longer sequence length makes distances comparable across journeys of different lengths.

What this shows
- A and B are slightly more similar than A and C by normalized edit distance.
- Different cost schemes, such as weighted costs, can change which journeys are considered closer.
- Edit distance compares sequences by modeling the cost of transforming the journey into another.

04 Dynamic programming — How do we find the minimum without trying every path?

Build the optimal solution from smaller subproblems.

The big idea: Dynamic programming finds the minimum edit cost between two journeys by filling a table of smaller subproblems.

04.0 Reading the DP table

Each cell $D(i, j)$ is the minimum number of edits needed to transform the first i events of A into the first j events of B.

A (4 events): S V C P (1 2 3 4)
B (5 events): S V X C P (1 2 3 4 5)

Columns → B (prefix length j)
Rows → A (prefix length i)

	∅	S	V	X	C	P
∅						
S						
V			$D(i, j)$			
C						
P						

Row i = first i events of A.
Column j = first j events of B.
Base cases: $D(i, 0) = i$, $D(0, j) = j$
$D(i, j)$ = minimum edits.

04.1 Compute D(2, 3) — where do the numbers come from?

First 2 events of A (S, V) vs. first 3 of B (S, V, X).
Consider the three possibilities from neighboring cells.

	∅	S	V	X	C	P
∅	0	1	2	3		
S	1	0	1	2		
V	2	1	0	1		
C						
P						

Three ways to reach D(2, 3):
Above: D(1, 3) + 1 = 2 + 1 = 3 (delete V)
Left: D(2, 2) + 1 = 0 + 1 = 1 (insert X)
Diagonal: D(1, 2) + 1 = 1 + 1 = 2 (V → X)

D(2, 3) = min(3, 1, 2) = 1

Insert Compare (X) after View.

04.2 Compute D(3, 4) — reuse a previous result

First 3 events of A (S, V, C) vs. first 4 of B (S, V, X, C).
Last events match (C = C), so reuse the diagonal.

	∅	S	V	X	C	P
∅	0	1	2	3	4	
S	1	0	1	2	3	
V	2	1	0	1	2	
C	3	2	1	1	1	
P						

Cost matches Cart: diagonal + 0.

D(3, 4) = D(2, 3) + 0 = 1

This reuses a solved prefix problem.

04.3 Completed matrix

After filling all cells, the bottom-right value gives the minimum edit distance between A and B.

	∅	S	V	X	C	P
∅	0	1	2	3	4	5
S	1	0	1	2	3	4
V	2	1	0	1	2	3
C	3	2	1	1	1	2
P	4	3	2	2	2	1

D(4, 5) = 1

One insertion transforms A into B.

04.4 Traceback from D(4, 5) to D(0, 0)

Follow the arrows backward to recover one optimal edit sequence.

	∅	S	V	X	C	P
∅	0	1	2	3	4	5
S	1	0	1	2	3	4
V	2	1	0	1	2	3
C	3	2	1	1	1	2
P	4	3	2	2	2	1

BACKWARD
Start: D(4,5)
Follow a valid predecessor.
Blue: match (0)
Ochre: insert (+1)

Path: (4,5) → (3,4) → (2,3) → (2,2) → (1,1) → (0,0)

04.5 Recurrence (Levenshtein distance)

The recurrence says each cell looks one step left, up, and diagonal, then keeps the cheapest explanation.

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 & \text{(deletion)} \\ D(i, j-1) + 1 & \text{(insertion)} \\ D(i-1, j-1) + \text{cost} & \text{(match or substitution)} \end{cases}$$

Delete from A, insert into A, or match/substitute the final event.

where **cost = 0 if A[i] = B[j]**, else 1.

04.6 Why dynamic programming works

Prefix: the opening portion of a sequence. For example, S V C is a three-event prefix of S V C P.

1 Optimal substructure
Definition: An optimal solution can be built from optimal solutions to smaller subproblems.
In edit distance: Once the final edit is chosen, the earlier prefix alignment must also have minimum cost.
Why it works: If a cheaper prefix alignment existed, we could replace the earlier steps and lower the total cost.

2 Overlapping subproblems
Definition: Different candidate solutions repeatedly encounter the same smaller subproblems.
In edit distance: Many edit paths need the same prefix comparison, such as D(2,3).
Why it matters: Compute each result once, store it, and reuse it instead of recomputing it.

04.7 Algorithmic view — bottom-up tabulation

1. **Initialize** — Fill the top row and left column.
2. **Fill by row** — Visit $i = 1..m$; for each j, visit $j = 1..n$.
3. **Apply recurrence** — Use the already-computed above, left, and diagonal.
4. **Read the answer** — D[m][n] is the distance; traceback recovers edits.

Verified Python · distance only

```
def levenshtein(a, b):
    m, n = len(a), len(b)
    d = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        d[i][0] = i
    for j in range(n + 1):
        d[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            c = 0 if a[i-1] == b[j-1] else 1
            d[i][j] = min(d[i-1][j] + 1,
                          d[i][j-1] + 1,
                          d[i-1][j-1] +c)
    return d[m][n]
```

04.8 Implementation styles

Memoization (top-down)
Start with the full problem. Solve smaller prefix problems recursively, caching each result when first computed.

Only requested subproblems are evaluated.

Tabulation (bottom-up)
Initialize base cases. Fill cells iteratively in dependency order, often row by row.

The resulting table can support traceback and inspection.

05 Longest common subsequence (LCS) — How do we find the shared sequence and understand what it means?

LCS ignores extra events and keeps relative order.

05.0 What is a subsequence and LCS?

A subsequence keeps the same relative order, but can skip characters. The longest common subsequence (LCS) is the longest sequence that appears in both strings.

A (4 events): S V C P
B (5 events): S V X C P
C (4 events): S V C E

A: SVCP
B: SVXCP
C: SVCE

For A and B, one LCS is *SVCP* (length 4).
For A and C, one LCS is *SVC* (length 3).

05.1 Same prefixes, different question

Compare the same prefixes: A = *SV* and B = *SVX*. Edit distance asks how to transform one into the other; LCS asks how much ordered structure they already share.

Edit distance $D(i, j)$
Minimum edits to transform $A[1..i] \to B[1..j]$

$D(2, 3) = 1$

A[1..2] S V
B[1..3] S V X

1 edit needed (insert *X*).

LCS $L(i, j)$
Length of the longest common subsequence

$L(2, 3) = 2$

A[1..2] S V
B[1..3] S V X

Longest shared subsequence: *SV* (length 2).

Different units, not competing scores: 1 edit versus 2 shared ordered events.

05.2 DP table snapshot 1 – mismatch at L(2, 3) = 2

Compute $L(2, 3)$ for A (S, V, C, P) vs. B (S, V, X, C, P).
The characters differ (V vs. X), so we take the maximum from left and up.

A/B	∅	S	V	X	C	P
∅	0	0	0	0	0	0
S	0	1	1	1	1	1
V	0	1	2	2	2	2
C	0	1	2	2	3	3
P	0	1	2	2	3	4

Characters at (2, 3)
A[2] = V
B[3] = X
(V ≠ X)

L(2, 3) = max(
L(1, 3) = 1 (up)
L(2, 2) = 2 (left)
) = 2

L(2, 3) = 2

The best subsequence for these prefixes is *SV*.

05.3 DP table snapshot 2 – match at L(3, 4) = 3

Compute $L(3, 4)$. The characters match ($C = C$), so we take the diagonal value and add 1.

A/B	∅	S	V	X	C	P
∅	0	0	0	0	0	0
S	0	1	1	1	1	1
V	0	1	2	2	2	2
C	0	1	2	2	3	3
P	0	1	2	2	3	4

Characters at (3, 4)
A[3] = C
B[4] = C
(C = C)

L(3, 4) = L(2, 3) + 1
= 2 + 1 = 3

L(3, 4) = 3

We extend the best subsequence (*SV*) with *C* to get *SVC*.

05.4 LCS recurrence

Let $L(i, j)$ be the length of the LCS between $A[1..i]$ and $B[1..j]$.

$$L(i, j) = \begin{cases} L(i-1, j-1) + 1 & \text{if } A[i] = B[j] \\ \max\left(L(i-1, j),\ L(i, j-1)\right) & \text{if } A[i] \neq B[j] \end{cases}$$

- If the characters match, we can extend the best subsequence from the diagonal by 1.
- If they don't match, we carry forward the best subsequence from either skipping a character from A or B.

Base cases: $L(i, 0) = 0$ and $L(0, j) = 0$.

05.5 What do the LCS results mean for our journeys?

A vs. B: all of A is preserved

$A = S\,V\,C\,P$ (4)
$B = S\,V\,X\,C\,P$ (5)
LCS = S V C P
Length = 4

- *B* has one extra event (*X*) that is skipped.
- *A* is fully contained in *B* as a subsequence.

A vs. C: diverge at the end

$A = S\,V\,C\,P$ (4)
$C = S\,V\,C\,E$ (4)
LCS = S V C
Length = 3

- Both reach Cart, but *A* purchases while *C* exits.
- They share 3 ordered events; the last event differs.

05.6 How should we normalize LCS?

LCS length alone does not tell us enough. We can normalize it in two different ways depending on what we want to measure.

Coverage (relative to longer) $\frac{L(m,n)}{\max(|A|, |B|)}$
What fraction of the longer sequence is covered by the shared subsequence?

Containment (relative to shorter) $\frac{L(m,n)}{\min(|A|, |B|)}$
How much of the shorter sequence is matched?

Pair	LCS	Coverage	Containment
A vs. B	4	4/5 = 0.80	4/4 = 1.00
A vs. C	3	3/4 = 0.75	3/4 = 0.75

Coverage measures shared structure relative to the longer journey; containment asks whether the shorter journey is fully contained in the longer one.

05.7 Relationship to edit distance

If we allow only insertions and deletions (no substitutions), every shared event in the LCS saves two edits. The distance is therefore the total sequence lengths minus twice the LCS length.

$$d_{\text{ins,del}}(A, B) = |A| + |B| - 2L(m, n)$$

A vs. B
$A = S\,V\,C\,P$ (4)
$B = S\,V\,X\,C\,P$ (5)
$4 + 5 - 2 \cdot 4 = 1$
With insertions and deletions only, the distance is 1 (insert *X*).

A vs. C
$A = S\,V\,C\,P$ (4)
$C = S\,V\,C\,E$ (4)
$4 + 4 - 2 \cdot 3 = 2$
With insertions and deletions only, the distance is 2 (delete *P* and insert *E*).

Standard Levenshtein distance can be smaller when substitutions are allowed; for example, *A* vs. *C* can cost 1 if we replace *P* with *E*.

06 Shingling + Jaccard — What changes when we compare local patterns?

Shingling compares sets of adjacent event pairs (n-grams), capturing local navigation behavior.

1. Extract bigrams (window size 2)
Slide a 2-event window to capture local transitions between consecutive events.

A: {SV, VC, CP}
B: {SV, VX, XC, CP}
C: {SV, VC, CE}

2. Compare using Jaccard similarity
Jaccard similarity divides the number of shared distinct bigrams by the total number of distinct bigrams across both journeys.

$$J(G_A, G_B) = \frac{|G_A \cap G_B|}{|G_A \cup G_B|}$$

where G_A and G_B are the sets of distinct bigrams.

A vs B
Intersection: {SV, CP}
Union: {SV, VC, VX, XC, CP}
J(A, B) = 2/5 = 0.40

A vs C
Intersection: {SV, VC}
Union: {SV, VC, CP, CE}
J(A, C) = 2/4 = 0.50

3. Information loss example
Different sequences can have the same set of bigrams, even though the full sequences are different.

X → Y → X → Y
Bigrams: {XY, YX}

Y → X → Y → X
Bigrams: {XY, YX}

Both have the same bigram set, but the overall sequences are different.

Takeaway
Shingling identifies shared local transitions even when whole journeys differ.
But it discards global sequence information, so it may miss important differences in the overall journey.

Result: different methods can give different answers

Levenshtein distance (smaller is more similar)
A – B = 1
A – C = 1
Tie

Longest common subsequence (larger is more similar)
A – B = 4
A – C = 3
B is more similar

Shingling + Jaccard (larger is more similar)
A – B = 0.40
A – C = 0.50
C is more similar

07 Method selection — Which measure should we use to compare our journeys?

Three common use cases

1 Find journeys with similar consequential changes
Question: Which journey requires the fewest meaningful changes from A?
Starting point: Weighted edit distance
A: Search → View → Cart → Purchase
B: Search → View → Compare → Cart → Purchase
C: Search → View → Cart → Exit
Unit Levenshtein distance (vs. A)
A → B = 1
A → C = 1
Both are one edit, but the consequences differ.
Choose this when the type and consequence of each transformation matter.
An extra Compare step may be acceptable; changing Purchase to Exit is more consequential. Assign weighted costs; the cheapest valid edit path determines the distance.

2 Find preserved purchasing progressions
Question: Which journey preserves A's purchasing progression despite detours?
Starting point: Longest common subsequence (LCS)
A: Search → View → Cart → Purchase
B: Search → View → Compare → Cart → Purchase
C: Search → View → Cart → Exit
Longest common subsequence with A
A: Search → View → Cart → Purchase (length 4)
B: Search → View → Cart → Purchase (length 4)
C: Search → View → Cart → (length 3)
Choose this when shared ordered milestones matter more than extra intermediate steps.
It's extra Compare doesn't break the purchasing progression. LCS favors B (4 vs. 3).

3 Find similar browsing patterns
Question: Which journey shares A's adjacent navigation transitions?
Starting point: Shingling + Jaccard
A: Search → View → Cart → Purchase
B: Search → View → Compare → Cart → Purchase
C: Search → View → Cart → Exit
Bigrams (adjacent transitions)

A (reference)	B	C
SV	SV	SV
VC	VX	VC
CP	XC	CE
	CP	

A vs. B
Shared: {SV, CP} (2)
Union: {SV, VC, CP, VX, XC} (5)
Jaccard = 2/5 = 0.40

A vs. C
Shared: {SV, VC} (2)
Union: {SV, VC, CP, CE} (4)
Jaccard = 2/4 = 0.50

Choose this when local transition patterns matter more than the full outcome trajectory.
C retains A's early transitions (Search → View and View → Cart), so Jaccard favors C, even though C exits instead of purchasing.

A five-step decision framework

	Decision	Guiding question	Applied to our example
1	Define the unit.	Are we comparing events, and where does it end?	Each such shopping session from first interaction to Purchase, Exit, or timeout. Don't merge unrelated visits.
2	Identify meaningful differences.	Which changes matter to the business?	An extra Compare step may be harmless; changing Purchase to Exit may indicate cart abandonment. Assign higher costs to more consequential changes.
3	Choose the objective.	What kind of similarity answers our question?	**Consequential changes:** weighted edit distance. **Ordered milestones:** LCS. **Local transitions:** bigram Jaccard.
4	Check scale and normalization	Are scores comparable across sessions?	One edit in a 4-step journey differs from a 40-step journey. Normalize by length, and for larger catalogs consider candidate retrieval.
5	Validate against use.	Do similar journeys serve our business goal?	For purchase analysis, check whether retrieved neighbors also purchase — not just share early browsing steps.

Remember
- Preserve the structure that matters.
- Define the differences that matter.
- Match the method to the task and scale.

Field Note
Mathematical closeness is not behavioral equivalence.
Journey C is one edit from A but exits instead of purchasing.
Journey B preserves A's purchasing progression, yet has lower bigram Jaccard similarity than C.
Similarity depends on what we measure.

linkedin.com/in/jaimesingson